

```
from abc import ABC, abstractmethod
```

```
# =====
```

```
# Продукт
```

```
# =====
```

```
class Phone(ABC):
```

```
    def __init__(self, brand: str, price: float, model: str, processor: str, ram: int):
```

```
        self.brand = brand
```

```
        self.price = price
```

```
        self.model = model
```

```
        self.processor = processor
```

```
        self.ram = ram
```

```
    @abstractmethod
```

```
    def get_info(self) -> str:
```

```
        pass
```

```
# =====
```

```
# Конкретные продукты
```

```
# =====
```

```
class Nokia(Phone):
```

```
    def __init__(self, brand: str, price: float, model: str, processor: str, ram: int,  
battery_working: bool):
```

```
        super().__init__(brand, price, model, processor, ram)
```

```
        self.battery_working = battery_working
```

```
    def get_info(self) -> str:
```

```
        battery_status = "Да" if self.battery_working else "Нет"
```

```
        return (
```

```
            f"Телефон: {self.brand}\n"
```

```
            f"Цена: {self.price} тг\n"
```

```
            f"Модель: {self.model}\n"
```

```
            f"Процессор: {self.processor}\n"
```

```
            f"Оперативная память: {self.ram} ГБ\n"
```

```
            f"Работает ли батарея: {battery_status}\n"
```

```
        )
```

```
class Samsung(Phone):
```

```
    def __init__(self, brand: str, price: float, model: str, processor: str, ram: int,  
front_camera_on: bool):
```

```
        super().__init__(brand, price, model, processor, ram)
```

```
        self.front_camera_on = front_camera_on
```

```
    def get_info(self) -> str:
```

```
        camera_status = "Да" if self.front_camera_on else "Нет"
```

```

return (
    f"Телефон: {self.brand}\n"
    f"Цена: {self.price} тг\n"
    f"Модель: {self.model}\n"
    f"Процессор: {self.processor}\n"
    f"Оперативная память: {self.ram} ГБ\n"
    f"Включена ли фронтальная камера: {camera_status}\n"
)

```

```

# =====
# Создатель
# =====
class iPhoneDeveloper(ABC):
    @abstractmethod
    def create_phone(self) -> Phone:
        pass

```

```

# =====
# Конкретные создатели
# =====
class NokiaDeveloper(IPhoneDeveloper):
    def create_phone(self) -> Phone:
        return Nokia(
            brand="Nokia",
            price=85000,
            model="Nokia G21",
            processor="Unisoc T606",
            ram=4,
            battery_working=True
        )

```

```

class SamsungDeveloper(IPhoneDeveloper):
    def create_phone(self) -> Phone:
        return Samsung(
            brand="Samsung",
            price=145000,
            model="Samsung Galaxy A24",
            processor="Helio G99",
            ram=6,
            front_camera_on=False
        )

```

```

# =====
# Клиентский код

```

```
# =====  
def client_code(developer: IPhoneDeveloper):  
    phone = developer.create_phone()  
    print(phone.get_info())  
    print("-" * 40)  
  
if __name__ == "__main__":  
    print("Создание телефона Nokia через фабричный метод:")  
    client_code(NokiaDeveloper())  
  
    print("Создание телефона Samsung через фабричный метод:")  
    client_code(SamsungDeveloper())
```

Создание телефона Nokia через фабричный метод:

Телефон: Nokia

Цена: 85000 тг

Модель: Nokia G21

Процессор: Unisoc T606

Оперативная память: 4 ГБ

Работает ли батарея: Да

Создание телефона Samsung через фабричный метод:

Телефон: Samsung

Цена: 145000 тг

Модель: Samsung Galaxy A24

Процессор: Helio G99

Оперативная память: 6 ГБ

Включена ли фронтальная камера: Нет

## Введение

В современном программировании большое внимание уделяется правильной организации кода и использованию шаблонов проектирования (design patterns). Они позволяют создавать гибкие, расширяемые и поддерживаемые приложения. Одним из таких шаблонов является **Фабричный метод (Factory Method)**, который относится к порождающим паттернам.

В рамках данного практического задания была реализована программа на языке Python, демонстрирующая применение паттерна «Фабричный метод» на примере создания объектов класса «Телефон». Основная цель работы — изучить принципы работы фабричного метода и научиться применять его на практике.

# Цель работы

Целью данного проекта является:

- изучение паттерна проектирования «Фабричный метод»;
- разработка иерархии классов на языке Python;
- реализация абстрактных классов и наследования;
- создание программы, позволяющей создавать различные объекты через единый интерфейс;

## Теоретическая часть

Паттерн «Фабричный метод» — это шаблон проектирования, который определяет интерфейс для создания объектов, но позволяет подклассам изменять тип создаваемых объектов.

Основная идея заключается в том, чтобы делегировать создание объектов специальным классам (фабрикам), а не создавать их напрямую в клиентском коде. Это делает систему более гибкой и позволяет легко добавлять новые типы объектов без изменения существующего кода.

Преимущества использования фабричного метода:

- уменьшение зависимости кода от конкретных классов;
- упрощение расширения программы;
- улучшение структуры и читаемости кода;
- соблюдение принципа открытости/закрытости (Open/Closed Principle).

## Заключение

В ходе выполнения практической работы был успешно изучен и реализован паттерн «Фабричный метод» на языке Python. Были освоены принципы работы с абстрактными классами, наследованием и полиморфизмом.

Разработанная программа демонстрирует, как можно гибко создавать объекты разных типов через единый интерфейс, что особенно важно при разработке крупных и масштабируемых систем.

Полученные знания могут быть применены в реальных проектах для улучшения архитектуры программ и повышения качества кода.